

Virtuaalne lemmikloom (Tamagotchi-stiilis)

Loore-Lotta Rahula, Lillii-Ann Older

Rühmatöö

Projekti kirjeldus

Programm imiteerib Tamagotchi mänguasja virtuaalses keskkonnas. Meie rühmatöö eesmärk on luua meelelahutuslik mäng, kus saad luua omale lemmiklooma ja tema eest hoolitseda.

Esimesena näeb kasutaja mängu juhust. Mängija saab valida, kas alustada uut mängu või mängida edasi eelnevalt salvestatud mängu.

Seejärel saab kasutaja luua lemmiklooma, sisestades looma nime ja vanuse ning valides liigi. On võimalus valida koera, jänese ja draakoni vahel. Iga looma puhul on ka vanuse limiit ning osad tegevused erinevad, et looma valik mängu mõjutaks ja seeläbi mäng põnevam oleks.

Kui lemmik on loodud, algab põhiline programmi tsükkel. Kasutajalt küsitakse, mis tegevust ta teha soovib - lemmikut ravida, süüa, pesta või temaga mängida. Graafilise kasutajaliidesega saab valida tegevusi nii hiirega nuppudele vajutades kui ka klaviatuuril tähtede ja nooltega liikudes.

Tegevustel on omakorda eraldi valikud, mis tõstavad või langetavad looma näitajaid, neid saab näha peale tegevuse valimist nooltega üles või alla liikudes.

Akna vasakul pool on näha pidevalt looma näitajad, mis vastavalt tegevustele ja juhuslikele sündmustele muutuvad. Näitajad on vahemikus 0-100 ning kui mõni neist jõuab 0 tasemeni, siis lemmik sureb ja mäng saab läbi. Iga 3 sammu tagant toimub ka juhuslik sündmus, mis muudab lemmiku näitajaid.

Mäng käib nii kaua, kuni lemmik sureb või mängija valib mängu lõpetada.

Kasutusjuhend

Projekti käivitamiseks tuleb see alla laadida githubist ning laadida enda arvutisse ka javafx library (link JavaFX library jaoks: [link](#); link mängule: [link](#)).

Pärast lingi allalaadimist tuleb navigeerida IntelliJ's File → Project Structure. "+" märgiga tuleb lisada New Project Library → Java. Sinna tuleb kopeerida oma arvutis link JavaFX lib kausta (TEE SINU KAUSTA\javafx-sdk-26.0.1\lib). Järgmiseks tuleb liikuda Run → Edit Configurations. "+" märgile vajutades tuleb lisada Main class → Peaklass, Modify Options otsida üles VM options ja see lisada ning sinna kopeerida vastav rida:

```
--module-path "TEE/SINU/JAVAFX/lib" --add-modules javafx.controls
```

Kui need konfiguratsioonid on lisatud, saab jooksutada koodi.

Mängu mängimiseks kuvatakse mängu käivitamise järel avaekraanil info seoses sellega, kuidas mäng on üles ehitatud ning kuidas mängida.

Klassid

Projektis on 8 klassi.

- Peaklass - JavaFX põhiklass, kogu graafiline kasutajaliides (main() meetod, mis käivitab programmi launch() abil). Seal luuakse kolm vaadet: avakuva tutvustusega ja mängu alustamise nuppudega, looma loomise vaade, kus küsitakse nime, liiki ja vanust ning kontrollitakse erindi abil sisendit ja viimaks põhimängu aken, kus on statistika, sündmuste logi ja tegevuste nupud. Polümorfismi kasutades luuakse vastavalt liigile õige Lemmikloom alamklassi isend. Klassis on kõik hiire- ja klaviatuurisündmused, ehk siin klassis luuakse võimalused mängida mängu nii klaviatuuri kui hiire abil ja tekitatakse hover-efektid jms. Pärast igat kasutaja tegevust uuendatakse graafiline kasutajaliides, et see näitaks õiget statistikat ja infot ning kontrollitakse, kas on aeg juhuslikuks sündmuseks.
- Lemmikloom - Abstraktne klass, mis määrab eri lemmikute ühised omadused (nimi, liik, vanus) ja näitajad (tervis, söömine, puhtus, meeleolu, mis on kõik vahemikus 0–100). Lisaks konstruktorile on olemas get ja set meetodid, et näitajaid muuta. Määrab ära, et loomade alamklassides peavad olema meetodid söö ja mängi ning getToiduValikud() ja getMänguValikud(). Klass on Serializable, seega saab terve objekti faili salvestada (binaarselt).
- Koer, Jänes ja Draakon - Lemmikloom klassi alamklassid. Nendes on meetodid söö() ja mängi(), sest nende puhul on vastavalt liigile tegevuste valikud erinevad (Eri loomad söövad ja mängivad erinevalt). Iga liik tagastab ka oma valikute nimekirja (getToiduValikud, getMänguValikud), mille põhjal graafiline kasutajaliides loob nupud. Mõned valikud on ka kahjulikud, näiteks koer ei tohi süüa šokolaadi ega jäätist, jänes ei talu šokolaadi ega mürkseeni, draakoni hambad murduvad teemandi söömisel ja vesi kustutab tema tule. Valede sisendite korral viskab klass SisendErind.
- Mänguhaldur - Klass, mis hoiab mängu olekut ja juhib mängu loogikat ning kus on loendur, mis loeb kasutaja tegevusi. Klassis on meetodid kahe tegevuse jaoks, mis toimuvad kõikidel liikidel ühtmoodi: ravi() ja pese(). Söömise ja mängimise loogika on aga igal loomal erinev, mistõttu need on alamklassides. Iga kasutaja tegevuse järel kutsutakse meetod lisaTegevus(), mis suurendab loendurit ja annab teada, kui on aeg juhuslikuks sündmuseks - iga kolme tegevuse järel. Juhuslik sündmus valitakse meetodis rakendaJuhuslikSündmus(), kus valitakse juhuslikult 10 võimaliku sündmuse hulgast üks (näiteks porilompi hüppamine, äikesetorm, herilase nõelamine, kosutav uni). Iga sündmus mõjutab erinevalt lemmiku näitajaid. Meetod

piiraStaatuseid() tagab, et kõik näitajad jäävad vahemikku 0–100. Kui mõni näitaja jõuab nullini, määrab see lemmiku surnuks ja mäng lõppeb.

- FailiHaldur - Klass selleks, et mäng salvestatakse ja hiljem laetakse binaarsest failist. Seal on ObjectOutputStream ja ObjectInputStream koos puhverdatud voogudega (BufferedOutputStream ja BufferedInputStream), mille abil saab terve Mänguhaldur objekti (koos lemmikloomaga) korruga kirjutada ja lugeda. Failioperatsioonid tehakse try-with-resources plokis. IOException ja ClassNotFoundException püütakse Peaklassis kinni ja kasutajale kuvatakse veateade.
- SisendErind - Erindiklass, selleks, kui kasutaja poolt sisestatud väärtused on valed. See visatakse igal pool, kus kasutaja sisend võib olla vale: tühi nimi, vale liik, vanus pole täisarv, vanus ületab liigi maksimumi, vale toidu- või mänguvaliku indeks. GUI-s püütakse erind kinni ja kuvatakse kasutajale arusaadav veateade punase tekstina või dialoogiaknas.

Projekti tegemise protsessi kirjeldus

Esimene etapp

Mõte tuli sellest, et tahtsime teha mängu, mis oleks jõukohane ja lõbus ning mida oleks põnev luua. See tundus täpselt õige olevat ja ühel rühmaliikmel on ka endal kodus tamagotchi stiilis mänguasi. Planeerisime algul, mis klasse vajame, mis meetodid neis on ja mis tüüpi need on. Mõtlesime ka ideid, mis edasiarenduseks teha saab ja mis tuleks kohe ära teha.

Kõigepealt lisasime hädavajalikud komponendid, lõime meetodid. Lõime peameetodisse alguse, et kasutaja saaks sisestada andmed ja luua lemmiklooma. Lisasime klassi vajalikud komponendid, et luua lemmik, väljastada info ja käivitada mängutsükkel. Tegime tsükli Mänguhaldur klassi, et mäng küsiks igal sammul kasutajalt tegevust ja väljastaks kolme korra tagant vajalikke andmeid.

Lõime alamklassid, et isenditel oleksid erinevad omadused ja saaksime luua eri klassi isendeid.

Lõpus lisasime kontrolli, kas lemmikloom on surnud ning lisasime juhuslikud sündmused.

Jooksvalt ka testisime ja *debug*-isime programmi.

Teine etapp - Graafiline kasutajaliides (GUI) ja edasiarendused

Teises etapis viisime mängu üle graafilisele kasutajaliidesele. Kasutasime selleks JavaFX-i ning GUI loomisel kasutasime abivahendina tehisintellekti (Claude AI, Opus 4.7). Tehisintellekti pakutud lahendused vaatasime ise üle ja mõtestasime läbi, et saaksime neid ka ise selgitada ning muuta.

Kuna GUI-s ei saa enam kasutada Scannerit, kohendasime kogu olemasolevat koodi: söömise ja mängimise meetodid võtavad sisendiks valiku numbri ning tagastavad tulemuse sõnumina, mida GUI saaks aknas kuvada. Mänguhaldurist eemaldasime konsoolitsükli ja jagasime loogika eraldi meetoditeks (ravi, pese, lisaTegevus), mida GUI saab vastavalt vajadusele välja kutsuda. Põhimängu tsükkel on nüüd GUI sündmuste põhine ehk iga nupuvajutus käivitab ühe tegevuse.

Lisasime ka mängu salvestamise ja laadimise faili (FailiHaldur klass), kasutades Serializable liidest ja binaarset I/O-d. Sellega saab kasutaja mängu salvestada ja pooleli jäänud mängu hiljem jätkata.

Erinditöötuse jaoks lõime oma erindiklassi SisendErind, mis tagab, et kasutaja vigased sisendid (tühi nimi, vale vanus, liigile sobimatu vanus jne) kuvavad selge veateate, millest kasutaja aru saab ja programm ei jookse kokku.

Igale loomale lisasime juurde mõned uued toidu- ja mänguvalikud, sh ka vastavalt liigile kahjulikud valikud (näiteks šokolaad koerale, mürkseen jänesele, teemant draakonile), et muuta mäng huvitavamaks ja sundida kasutajat teadlikumalt valima.

Lisasime klaviatuuri kiirklahvid (R, T, P, M, S) ning hover-efektid nuppudele, et programm reageeriks nii hiirele kui klaviatuurile.

Jooksvalt testisime ja parandasime probleeme, mis tekkisid. Näiteks emojid, mida programm ei suutnud õigesti kuvada, või klaviatuuri sündmused, mis alguses ei töötanud.

Programmi testimise protsess

Katsetasime põhiprogrammi tööd sisestades erinevaid nimesid ja vanuseid ning valides eri liike.

Mängisime programmi läbi eri loomade alamklassidega ja testisime, kas lemmik saab õigel hetkel surma või kas tema näitajad liiguvad õigesti. Vaatasime, et programm ei väljastaks üleliigset infot ja näitaks õigel ajal juhuslikke sündmuseid ja näitajaid ning kuvaks erindiga veateateid. Samuti vaatasime, et GUI näitaks emojisid ja hover efektid ning nupud ja klaviatuurisündmused töötaksid. Selleks proovisime, kas nupud reageerivad klaviatuuril nii väikeste kui ka suurte tähtedega ja saaks nooltega liikuda läbi valikute ja kas hiirega saab samuti nuppe kasutada. Veel vaatasime, et aknad liiguksid dünaamiliselt. Viimaks testisime, kas mängu salvestamisel ja akna kinni panemisel saame hiljem mängu salvestatud kohast jätkata.

Rollid rühmatöös

Projekti tegemise ajaks saime kokku ning arutasime läbi võimalikud lahendused ja ideed ning kuidas neid realiseerida. Kui üks kirjutas/täiustas koodi, siis teine jälgis kõrvalt, andis soovitusi ja aitas parandustega. Teise etappiga läks umbkaudselt 8 tundi (idee genereerimisest koodi valmimiseni).

Murekohad

Peamiseks mureks oli kasutajaliidese loomine, mille ise kujundamine oleks võtnud päris pika aja.

Õnneks saime AI abil loodud esindusliku liidese. Kuid kuna AI loodud lahendustes olid osalised grammatilised viperused sees, vaatasime projekti kriitilise pilguga üle, et need parandada ning et ka kõik vastavad emotikonid mängu siseselt töötaksid.

Hinnang tööle

Oleme tulemusega rahul, kuna saime valmis töötava ning visuaalselt hea välimusega mängu.

Funktsionaalsused töötavad ning mäng on lihtsalt mõistetav ja mängitav.